OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 10

Team No.: 19

Project Title: Linux File Descriptor and File I/O Management System

Team Members

Roll Number	Student Name	Signature
2520080055	N.YEGNASAIREDDY	
2520090036	SURESH BABU	
2520090204	SONU PARWAR	

1. Abstract

The **Linux File Descriptor and File I/O Management System** is a Linux-based system programming project designed to demonstrate how files are created, opened, read, written, and closed using Linux system calls. The project provides a simple interface for performing basic file operations and understanding how file descriptors are used by the operating system to manage open files. It uses system calls such as open(), read(), write(), lseek(), and close() to perform file input and output operations. The system will allow users to create or open a file, display its contents, add new data, overwrite existing data, and close the file safely. It will also display important information such as the file descriptor returned by the operating system. The project demonstrates important Operating Systems and System Programming concepts, including file descriptors, file handling, system calls, kernel-level file management, and standard input/output. The expected outcome is a simple and user-friendly Linux application that clearly demonstrates how Linux manages files and file descriptors.

2. Problem Statement

In Linux, file operations are managed by the operating system using **file descriptors and system calls**. Understanding how these mechanisms work can be difficult for beginners because file operations normally happen behind high-level programming interfaces.

This project aims to develop a simple Linux-based system that demonstrates file creation, opening, reading, writing, seeking, and closing using low-level Linux system calls. It will help students understand how the operating system manages files through file descriptors and how applications communicate with the Linux kernel.

3. Objectives

- **To understand and demonstrate Linux file descriptors and their role in file management.**
- **To implement file operations using Linux system calls such as open(), read(), write(), and close().**
- **To provide a simple menu-based interface for creating, reading, writing, and managing files.**
- **To demonstrate how Linux applications communicate with the operating system kernel for file I/O operations.**

4. Proposed Methodology

The project will be developed in **Linux/Ubuntu using C programming**. A menu-driven application will be created to allow the user to perform different file operations.

The program will use the open() system call to open or create files and obtain a file descriptor. The read() and write() system calls will be used to read data from and write data to files. The lseek() system call will be used to change the file position, and close() will release the file descriptor after completing the operation.

The basic flow will be:

Start → Display Menu → Select File Operation → Perform System Call → Display Result → Continue/Exit

5. Operating Systems Concepts / Linux APIs Used

(List the relevant Operating Systems concepts, Linux system calls, POSIX APIs, or mechanisms that will be used.)

OS Concept / Linux API / System Call	Purpose in the Project
System Calls	Provides communication between application and Linux kernel
File I/O	Performs input and output operations
File Descriptor	Identifies an opened file
open()	Opens or creates a file
read()	Reads data from a file
write()	Writes data to a file
lseek()	Changes the file position
close()	Closes the file descriptor
Kernel File Management	Manages files and file descriptors

6. Individual Contribution

(Clearly specify the responsibility of each team member in the project.)

Roll Number	Student Name	Individual Responsibility
2520080055	N.yegnasaireddy	File opening, creation and file descriptor implementation
2520090036	Suresh babu	Read and write system call implementation
2520090204	Sonu parwar	Menu interface and testing

7. Tools / Platforms / Software Used

(List the programming languages, operating system, libraries, tools, and software planned for the project.)

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating system and execution environment
C / C++	Project development
GCC Compiler	Compiling the C program
VS Code / Vim	Writing and editing source code
Linux Terminal	Compiling and executing the project
GitHub	Project source-code management and submission

8. Expected Outcome

The project is expected to produce a simple Linux-based file management application that allows users to create, open, read, write, seek, and close files using Linux system calls. The application will display the file descriptor and the result of each operation.

The project will provide practical understanding of **file descriptors, low-level file I/O, system calls, and Linux kernel interaction**. It will also demonstrate how applications communicate with the operating system for managing files.

9. GitHub Repository

GitHub Repository Link: https://github.com/yegnasai/2520080055_ossp/tree/KLH-%3CCSIT&AIDS%3E-%3C2026%3E-%3CTEAMNO19%3E-%3C-Linux-File-%3E

Repository Created: YES

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: _____M.RAGHUPATHI____

Signature: ____

Remarks: ____

Faculty Use

Project Approved: ____________________________________

☐ Yes ☐ No

Date: ____

Faculty Signature: ____